

2022年度（2022年10月・2023年4月入学）入学試験問題 解答 ― 大問 3

千葉大学大学院融合理工学府 数学情報科学専攻 情報科学コース 専門科目／C言語：加算・シフト・減算による非負整数の乗算

問1 繰り返し加算による乗算（図3-1 mult1）

x を y 回足し込めば $x \times y$ になる。 z に x を加えながら y を 1 ずつ減らす。

(1) 空欄1～3

空欄	解答	意味
1	<code>y--</code>	条件 $y-- > 0$: 今の y で判定後に y を1減らす → ちょうど y 回ループ
2	<code>z +=</code>	$z += x$; 1回の加算で x を足し込む
3	<code>return z</code>	結果 $z (= x \times y)$ を返す

```
unsigned mult1(unsigned x, unsigned y)
{
    unsigned z = 0;

    while (y-- > 0) {
        z += x;    /* この行に注目 */
    }
    return z;
}
```

空欄2・3は問2（図3-2）と共通。図3-2で $z += (x << i)$; として使われることと整合する。ループ本体が1文だけなので、 y の減算は条件式 $y-- > 0$ の後置デクリメントで行う。

(2) 「この行に注目」の実行回数

`y` 回

`while (y-- > 0)` は元の y の値だけ本体を実行するので、`z += x`; は y 回実行される。

問2 シフトと加算による乗算（図3-2 mult2）

y の各ビット i を調べ、ビットが1なら x を i 桁ずらした $x \ll i$ ($= x \cdot 2^i$) を z に足す。2進筆算と同じ「シフト&加算」方式。

(1) 空欄4・5

空欄	解答	意味
4	<code>sizeof(unsigned)</code>	$8 \times \text{sizeof}(\text{unsigned}) = 8 \times 4 = 32 = \text{unsigned}$ の総ビット数（全ビットを走査）
5	<code>(1 << i)</code>	$y \ \& \ (1 \ll i)$ で y の第 i ビットが立っているか判定

```
unsigned mult2(unsigned x, unsigned y)
{
    unsigned z = 0;
    unsigned i;
```

```

for (i = 0; i < 8 * sizeof(unsigned); i++) {
    if (y & (1 << i)) {
        z += (x << i);    /* この行に注目 */
    }
}
return z;
}

```

空欄4は sizeof(y) や sizeof(x) でも可 (いずれも 4 バイト)。空欄5は 1u << i でも可。

(2) y=13~16 のときの実行回数 (= y の1のビット数)

この行は y の立っているビット (1のビット) の個数だけ実行される。

y	2進表記	1のビット数	実行回数
13	1101	3	3回
14	1110	3	3回
15	1111	4	4回
16	10000	1	1回

(3) 途中で処理を終えるための修正

上位ビットがすべて0になれば、それ以降の加算は決して起こらないので打ち切れる。(y >> i) が 0 になった時点で「未処理の上位ビットはすべて0」を意味する。

for文の継続条件に (y >> i) != 0 を加える

例: for (i = 0; i < 8*sizeof(unsigned) && (y >> i) != 0; i++)
(または、ループ先頭に if ((y >> i) == 0) break; を挿入)

こうすると、y の最上位の1のビットまで走査した時点でループを抜けられる。例えば y=1 なら 1 回で終了し、全32ビットを無駄に調べずに済む。

問3 減算も使って演算回数を減らす

減算を許すと、例えば 7倍 = 8倍 - 1倍 のように少ない回数で作れる。0 から始めて「2^k倍を加算/減算」を繰り返し、目的の n倍を作るのに必要な最小の加減算回数を考える。

(1) 13~16倍に必要な最小の加減算回数

0 から「±2^k倍」を足し引きして n倍を作る最小回数 = 符号付き2進 (NAF) 表現での非ゼロ桁の個数。

n倍	最小の表し方	最小回数
13	16 - 4 + 1 (= 2 ⁴ -2 ² +2 ⁰ 。2回では表せない)	3回
14	16 - 2 (= 2 ⁴ -2 ¹)	2回
15	16 - 1 (= 2 ⁴ -2 ⁰)	2回
16	16 (= 2 ⁴ 、シフトのみ+1回の加算)	1回

13 は加算のみだと 8+4+1 で3回。減算を使っても 16-2-1 等やはり3回で、2回には縮まない (2のべき乗の差・和で13を2項では作れない)。一方 14・15 は最上位の1つ上の2のべき乗から引く形で2回に減る。

(2) 漸化式 $C(n)$ (空欄6~10)

空欄	解答	意味
6	2^i (2のべき乗)	シフトだけで作れるのは 2のべき乗倍
7	1	$n=2^i$ なら 「iビットシフトして加算」 の1回でよい
8	$0 < j < 2^i (i = \lfloor \log_2 n \rfloor)$	2^i は n を超えない最大の2のべき乗、 j はその余り
9	$1 + C(j)$	加算法: 2^i 倍を1回で作る、残り j 倍を $C(j)$ 回で作る
10	$1 + C(2^{i+1} - n)$	減算法: 2^{i+1} 倍を1回で作る、 $(2^{i+1}-n)$ 倍を引く

$$C(n) = 1 \quad (n = 2^i)$$

$$C(n) = \min(1 + C(j), 1 + C(2^{i+1} - n)) \quad (n = 2^i + j, \quad 0 < j < 2^i)$$

検算: $C(15) = \min(1 + C(7), 1 + C(1)) = \min(1 + 2, 1 + 1) = 2$ ✓。 $C(13) = \min(1 + C(5), 1 + C(3)) = \min(1 + 2, 1 + 2) = 3$ ✓。
 $C(14) = \min(1 + C(6), 1 + C(2)) = \min(1 + 2, 1 + 1) = 2$ ✓。(1)の結果と一致。

(3) count 関数 (空欄11~13、図3-3)

まず while ループで i を 「 y のビット長」 ($2^{i-1} \leq y < 2^i$ となる i) まで進める。最大の2のべき乗は 2^{i-1} 。

空欄	解答	意味
11	$(1 \ll ++i) \leq y$	$2^i \leq y$ の間 i を進める。終了時 $i = y$ のビット長 ($2^{i-1} \leq y < 2^i$)
7	1	最上位項を作る1回ぶん ((2)と同じ)
12	$y - (1 \ll (i - 1))$	加算法の残り $j = y - 2^{i-1}$
13	$(1 \ll i) - y$	減算法の残り $2^i - y$

```
unsigned count(unsigned y)
{
    unsigned i = 0;
    unsigned plus, minus;

    if (y == 0) return 0;
    while ((1 << ++i) <= y)
        ;
    plus = 1 + count(y - (1 << (i - 1)));
    if (plus <= 2) return plus;
    minus = 1 + count((1 << i) - y);
    return plus <= minus ? plus : minus;
}
```

if (plus <= 2) return plus; が正しい枝刈りである理由: ループ後 $y < 2^i$ なので $2^i - y \geq 1$ 、よって $\text{minus} = 1 + \text{count}(2^i - y) \geq 2$ 。したがって $\text{plus} \leq 2$ のときは $\text{plus} \leq \text{minus}$ が保証され、 minus を計算せず plus を返してよい。
 検算 $y=13$: $i=4$ (13のビット長)、 $\text{plus} = 1 + \text{count}(13 - 8) = 1 + \text{count}(5) = 1 + 2 = 3$ 、
 $\text{minus} = 1 + \text{count}(16 - 13) = 1 + \text{count}(3) = 1 + 2 = 3 \rightarrow 3$ ✓。

(4) シフト量と加算/減算の別も返すための修正方針

演算回数だけでなく、各演算の「シフト量」と「加算か減算か」の並び (操作列) も返すように変更する。

考え方：

- 戻り値を「演算回数」と「操作列（各要素は (シフト量, 符号+/-) の組）」の組にする（構造体や配列+長さで表す）。
- **plus（加算法）**を採用した場合：先頭に「(i-1) ビットシフトして**加算**」を記録し、続けて残り $j = y - 2^{i-1}$ に対する再帰結果の操作列を**符号そのまま**で連結する。
- **minus（減算法）**を採用した場合：先頭に「i ビットシフトして**加算**」を記録し、続けて残り $2^i - y$ に対する再帰結果の操作列の**各符号を反転**（加算 $\leftrightarrow$ 減算）して連結する（その部分全体を引くため）。
- 呼び出し時の引数 = 「これから作るべき倍率」、戻り値の操作列 = 「0 から順に適用すべき (シフト量, $\pm$) の手順」。最終的にこの手順どおり $z \pm (x \ll \text{シフト量})$ を実行すれば $x \times y$ が得られる。

例： $y=15 \rightarrow$ 操作列 [(4, +), (0, -)], すなわち $z = (x \ll 4) - x$ 。 $y=13 \rightarrow$ [(4,+),(2,-),(0,+)], すなわち $z = (x \ll 4) - (x \ll 2) + x$ (= 16-4+1 倍)。

(5) この乗算法が有効になる状況

乗数 y がコンパイル時に決まっている定数の場合に有効。

理由：y が定数なら、(3)(4) の条件判定・再帰を**事前に（コンパイル時に）済ませて操作列を確定**でき、実行時には判定や分岐が不要になる。あとは固定された「シフト+少数回の加減算」だけを実行すればよく高速。特に次の状況で効果的：

- 定数 y が **2のべき乗 $\pm$ 少数項**で表せる（NAF表現の非ゼロ桁が少ない）とき。例： $\times 15 = (x \ll 4) - x$ はシフト1回+減算1回で済む。
- プロセッサが**乗算命令を持たない／乗算が遅い**場合。シフトと加減算は高速なので相対的に有利。

逆に y が実行時まで不明な一般の変数だと、毎回 (3) のような条件判定・再帰のコストがかかり、専用乗算命令に勝てない。「乗数が定数」という条件を絞れたときに初めて高速化の価値が出る。